SOFTWARE REQUIREMENTS SPECIFICATION

for

CampusSwap

Hostel Room Exchange System

Version 1.0

Prepared by

Jahnavi (1024240152)
Ronit Saini (1024240102)
Ranvir (1024240001)
Krishna Kapoor (1024240010)

Submitted to

[Teacher's name]
[Designation, e.g. Assistant Professor]
[Department]

UCS503P Software Engineering (Project), 2026–27 Odd Semester
Thapar Institute of Engineering and Technology, Patiala

October 9, 2026

Revision History

Name	Date	Reason for changes	Version
CampusSwap team	October 9, 2026	First complete version. Covers the working prototype (marked **Built**) and Phase 2 (marked **Planned**).	1.0

Contents

Revision History 2

List of Figures 5

List of Tables 6

1. Introduction 7
- 1.1. Purpose . . . 7
- 1.2. Document Conventions . . . 7
- 1.3. Intended Audience and Reading Suggestions . . . 7
- 1.4. Project Scope . . . 8
- 1.5. References . . . 8

2. Overall Description 10
- 2.1. Product Perspective . . . 10
- 2.2. User Classes and Characteristics . . . 10
- 2.3. Product Functions . . . 10
- 2.4. Operating Environment . . . 13
- 2.5. Design and Implementation Constraints . . . 13
- 2.6. Design . . . 15
  - 2.6.1. Student activities . . . 15
  - 2.6.2. Hostel staff (warden) activities . . . 15
  - 2.6.3. How a request and a chain change status . . . 15
  - 2.6.4. How the parts work together for one swap . . . 15
- 2.7. User Documentation . . . 16
- 2.8. Assumptions and Dependencies . . . 16

3. External Interface Requirements 20
- 3.1. User Interfaces . . . 20
- 3.2. Hardware Interfaces . . . 24
- 3.3. Software Interfaces . . . 24
  - 3.3.1. Web API . . . 24
  - 3.3.2. Internal interfaces . . . 25
- 3.4. Communications Interfaces . . . 25

4. System Features 26
- 4.1. Description and Priority . . . 26
- 4.2. Functional Requirements . . . 26
  - 4.2.1. Sign in and sessions . . . 26
  - 4.2.2. Browse rooms . . . 27
  - 4.2.3. Submit and withdraw a swap request . . . 28
  - 4.2.4. Run a matching round . . . 29
  - 4.2.5. Accept or decline a swap chain . . . 30
  - 4.2.6. Approve a chain . . . 31
  - 4.2.7. Status tracking and reports . . . 31

4.2.8. Faculty slot exchange (Phase 2) 32

5. Other Nonfunctional Requirements **33**
5.1. Performance Requirements 33
5.2. Safety Requirements 33
5.3. Security Requirements 34
5.4. Software Quality Attributes 34
5.5. Business Rules 35

6. Other Requirements **36**
6.1. Maintenance 36
6.2. Data and privacy 36
6.3. Deployment (Phase 2) 36

A. Glossary **37**

B. Analysis Models **38**

C. Traceability Matrix **42**

D. To Be Determined List **45**

List of Figures

1.1. Context diagram (data flow diagram, Level 0) 9

2.1. System architecture 11
2.2. Use case diagram 12
2.3. Data flow diagram, Level 1 14
2.4. Activity diagram: life of one swap request 16
2.5. State diagram: swap request 17
2.6. State diagram: swap chain 17
2.7. Sequence diagram: one swap from request to approval 18

3.1. Sign-in page 20
3.2. Explore Rooms page 21
3.3. Student dashboard 21
3.4. Swap ring on the student dashboard 22
3.5. Student dashboard on a phone 22
3.6. Hostel staff page 23
3.7. Engine trace and TTC versus one-to-one comparison 23

B.1. Class diagram, part 1: domain model 39
B.2. Class diagram, part 2: services and matching engine 40
B.3. ER diagram 41

List of Tables

2.1. User classes 10
2.2. Technology used and why 13

3.1. Web API 24
3.2. Error responses 25

4.1. Requirements: sign in and sessions 26
4.2. Requirements: browse rooms 27
4.3. Requirements: submit and withdraw a swap request 28
4.4. Requirements: run a matching round 29
4.5. Requirements: accept or decline a swap chain 30
4.6. Requirements: approve a chain 31
4.7. Requirements: status tracking and reports 32
4.8. Requirements: faculty slot exchange 32

5.1. Performance requirements 33
5.2. Safety requirements 33
5.3. Security requirements 34
5.4. Business rules 35

C.1. Requirements to code and tests 42

1. Introduction

1.1. Purpose

Students in a hostel often want to move to a different room: closer to friends, on a lower floor, with an attached washroom or with air conditioning. Today they post offers in WhatsApp groups and hope that someone wants the opposite swap. A direct swap needs two students who each want the other's room, which is rare. Longer chains, where A takes B's room, B takes C's and C takes A's, would satisfy many more students, but nobody can find them by hand. The warden's office, meanwhile, works with paper forms and has no central record of who asked for what.

CampusSwap solves this. Students rank up to five rooms they would prefer to their own. The hostel staff runs a matching round, and the system uses the *Top Trading Cycles* (TTC) method to find every chain of swaps in which each person moves to a room they ranked. Everyone in a chain must confirm, and then the warden approves the swap.

This document specifies the requirements for CampusSwap version 1.0. It says what the system must do (functional requirements), how well it must do it (nonfunctional requirements), and the rules it must follow. It covers both the working prototype and the planned second phase, and marks clearly which is which.

1.2. Document Conventions

- **Requirement IDs.** Every requirement has a unique ID such as `FR-REQ-05`. `FR` means functional requirement, `NFR` means nonfunctional requirement, and `BR` means business rule. The middle part names the feature (for example `REQ` for swap requests).
- **"Shall"** marks something the system must do.
- **Priority.** *High*: the system is not usable without it. *Medium*: important, but there is a work-around. *Low*: nice to have.
- **Status.** **Built** means the prototype already does it and an automated test or a screenshot in this document shows it. **Planned** means it is part of Phase 2 and is not built yet.
- `Monospace text` is a name taken from the code, such as an error code (`own_room`), a file or an API path.
- Technical terms are explained in the Glossary (Appendix A).

1.3. Intended Audience and Reading Suggestions

Reader	What to read
Course instructor and examiners	Chapters 1 and 2 give the big picture; Chapter 4 lists every requirement with its status.
Developers	Chapters 2–4 and the analysis models in Appendix B.

Reader	What to read
Testers	Chapter 4 and the traceability table in Appendix C, which links each requirement to the test that checks it.
Hostel office (future users)	Sections 1.4, 2.2 and 5.5, and the screens in Section 3.1.

1.4. Project Scope

CampusSwap is a web application for one institute's hostels. It gives students and hostel staff a shared place to manage room-swap requests from start to finish.

In scope:

- Signing in as a student or as hostel staff (the warden).
- Browsing the rooms a student is allowed to live in, and ranking up to five of them.
- Saving the ranked list as a swap request with a tracking ID.
- Running a matching round with Top Trading Cycles, and comparing it with direct one-to-one swaps on the same requests.
- Showing each student their swap chain, and collecting every member's confirmation within 48 hours.
- Warden approval, after which the students' rooms change in CampusSwap's records.
- Dashboards, an activity log and a spreadsheet (CSV) export.

Out of scope: first-time room allotment, hostel fees, mess management, and the hostel office's official master records. CampusSwap produces an approved swap record; the hostel office still updates its own records.

Goals. The project proposal defines first-phase success as *accepting a request, saving it to the database, running a match and showing the result.* The main measure of quality is the **Match Yield**: the share of students in a round who are placed in a swap. The target is at least 60% on a pool of 100 requests.

Figure 1.1 shows CampusSwap as a single system and the data that flows between it and its two kinds of users.

1.5. References

[1] IEEE, *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998.
[2] K. Wiegers and J. Beatty, *Software Requirements*, 3rd ed., Microsoft Press, 2013.
[3] L. Shapley and H. Scarf, "On cores and indivisibility," *Journal of Mathematical Economics*, vol. 1, no. 1, pp. 23–37, 1974.
[4] A. E. Roth and A. Postlewaite, "Weak versus strong domination in a market with indivisible goods," *Journal of Mathematical Economics*, vol. 4, no. 2, pp. 131–137, 1977.
[5] A. E. Roth, "Incentive compatibility in a market with indivisible goods," *Economics Letters*, vol. 9, no. 2, pp. 127–132, 1982.
[6] CampusSwap team, *CampusSwap: Project Proposal*, UCS503P, Thapar Institute of Engineering and Technology, 2026.
[7] CampusSwap source code and documentation: https://github.com/ranvir7123/UCS503P-202627-CampusSwap.

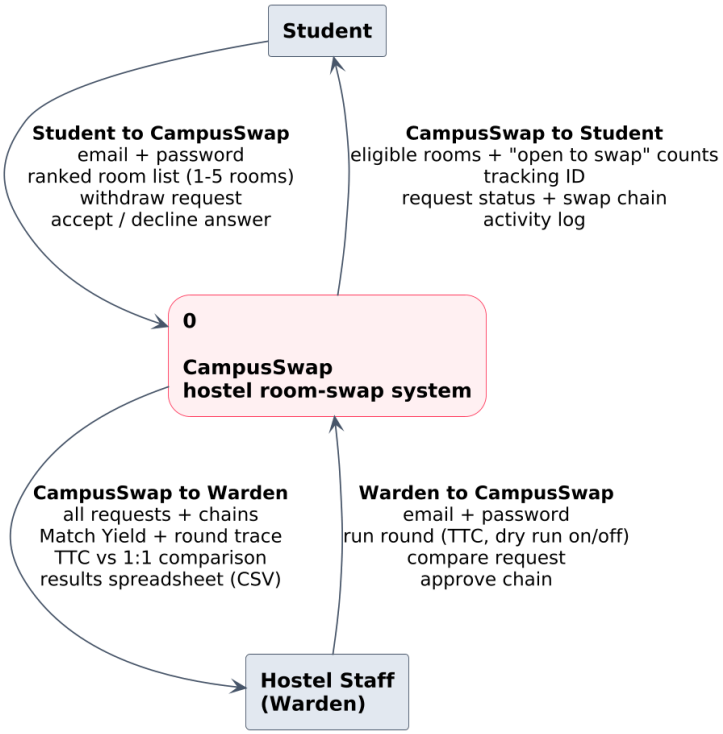


Figure 1.1.: Context diagram (data flow diagram, Level 0). The rounded box is the whole system; the square boxes are the people outside it. Each arrow lists the data that moves in that direction.

[8] FastAPI documentation: https://fastapi.tiangolo.com.
[9] SQLAlchemy 2.0 documentation: https://docs.sqlalchemy.org/en/20/.

2. Overall Description

2.1. Product Perspective

CampusSwap is a new, self-contained product. It replaces two informal processes: swap offers posted in WhatsApp groups, and paper request forms at the hostel office. It does not replace the hostel office's own allotment records (see business rule `BR-10`).

The system has three layers and a separate matching engine (Figure 2.1):

- **Pages** in the web browser show data and send the user's actions to the server. They contain no business rules.
- **The server** checks who is signed in, applies every rule, and stores data in a database.
- **The matching engine** runs Top Trading Cycles. It knows nothing about the database or the web, so it can be tested on its own with made-up data and reused later (for example, for exchanging faculty lecture slots).

2.2. User Classes and Characteristics

CampusSwap has two kinds of users. Figure 2.2 shows what each one can do.

Table 2.1.: User classes

User class	How many	Characteristics and needs
Student	Hundreds per hostel; most important class	Already lives in a hostel room (owns one bed, called a *seat*). Usually uses a phone. Not technical. Wants a better room, wants to know whether anything is happening, and wants to keep their current room if no good swap exists. Uses the system a few times a semester.
Hostel Staff (Warden)	One office per group of hostels	Runs matching rounds, checks the results, approves confirmed swaps and exports records. Uses a laptop. Needs a clear record of who asked for what and why each swap was proposed.
Visitor (not signed in)	Anyone	Sees only the sign-in page and its public numbers (open requests, rooms, last round). Sees no personal data.

2.3. Product Functions

The main functions are:

1. **Sign in** as a student or as hostel staff.

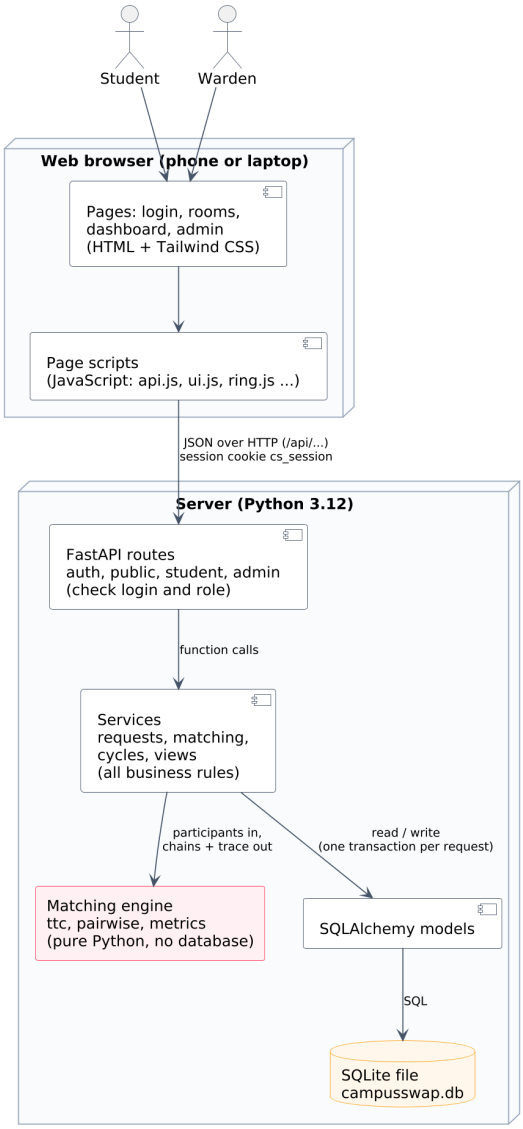


Figure 2.1.: System architecture. Each layer talks only to the layer next to it. The matching engine is pure Python and has no database code.

2. **Browse rooms** in the hostels the student may live in, with filters and a count of how many occupants of each room want to swap.
3. **Submit or withdraw a swap request**: a ranked list of one to five rooms, saved with a tracking ID such as `CS-2026-0013`.
4. **Run a matching round** (TTC) over every open request, see the chains it found, the Match Yield and a step-by-step trace, and compare the result with direct one-to-one swaps.
5. **Accept or decline** a proposed swap chain within 48 hours.
6. **Approve** a chain that every member confirmed, which moves each student to their new room.
7. **Show status and reports**: the student dashboard, the warden's overview and request table, and a CSV export.

Figure 2.3 breaks the system into these seven processes and shows the data each one reads and writes. The system keeps five groups of data (the data stores D1 to D5). Their full structure is in the ER diagram (Figure B.3).

- **D1 Users and sessions**: every account (email, name, password hash, role; students also have roll number, gender, year and branch) and every signed-in browser.

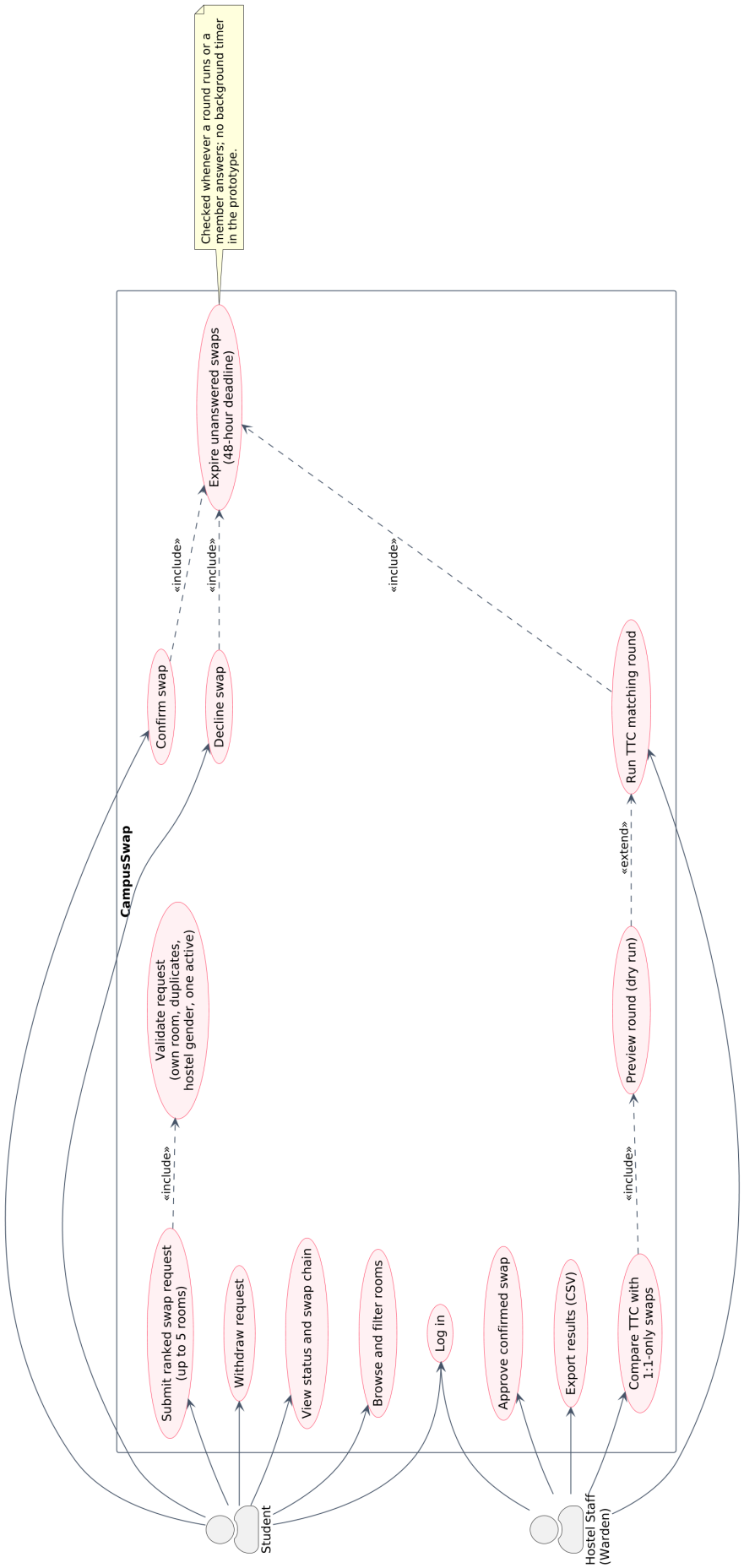


Figure 2.2.: Use case diagram. «include» marks a step that always happens as part of another use case; «extend» marks an optional variation.

- **D2 Hostels, rooms and seats**: each hostel's code, name and the gender it houses; each room's block, floor, number, number of beds (1–3), air conditioning, washroom type and tags; each bed (seat) and who sleeps in it.
- **D3 Swap requests and preferences**: the tracking ID, the student, the seat they offer, the status, and the ranked rooms.
- **D4 Match runs and swap chains**: each round's settings and results (pool size, matched, Match Yield, trace), each chain with its status and deadline, and each member's answer.
- **D5 Event log**: a time-stamped line for every important change, used for the activity list and the audit trail.

2.4. Operating Environment

- **Client:** any current web browser with JavaScript turned on, on a phone, tablet or laptop. The pages were checked in Microsoft Edge at a desktop width of 1440 pixels and a phone width of 390 pixels.
- **Server:** any computer that runs Python 3.12 or newer (Windows, Linux or macOS). By default the server listens on `127.0.0.1`, port `8000`; both can be changed with `-host` and `-port`.
- **Database:** one SQLite file, `campusswap.db`. Nothing else needs to be installed.
- **Network:** the pages work without internet access. Fonts, icons and style sheets are stored with the application, not loaded from other websites.

2.5. Design and Implementation Constraints

Table 2.2.: Technology used and why

Part	Choice and reason
Server	Python 3.12 with FastAPI 0.141.1, run by Uvicorn 0.52.4. Short, readable code, and FastAPI generates an interactive API page at `/docs`.
Database	SQLite through SQLAlchemy 2.0.52. A real SQL database in one file; moving to PostgreSQL only needs a different database address.
Pages	Plain HTML and JavaScript, styled with Tailwind CSS 3.4.19 built on the developer's computer, so no internet is needed during a demonstration.
Matching	Top Trading Cycles, as promised in the proposal, written in plain Python with no database code.
Testing	pytest 9.1.1 and httpx 0.28.1; GitHub Actions runs every test on each push and pull request.
Diagrams	PlantUML text files in `diagrams/`, drawn into images by `diagrams/render.py`.

Other constraints:

- **No real student data** may be used in a class project. All hostels, names, roll numbers and email addresses in the prototype are made up.
- The institute's single sign-on (Microsoft account) cannot be used without the institute's approval, so the prototype uses its own email and password sign-in.

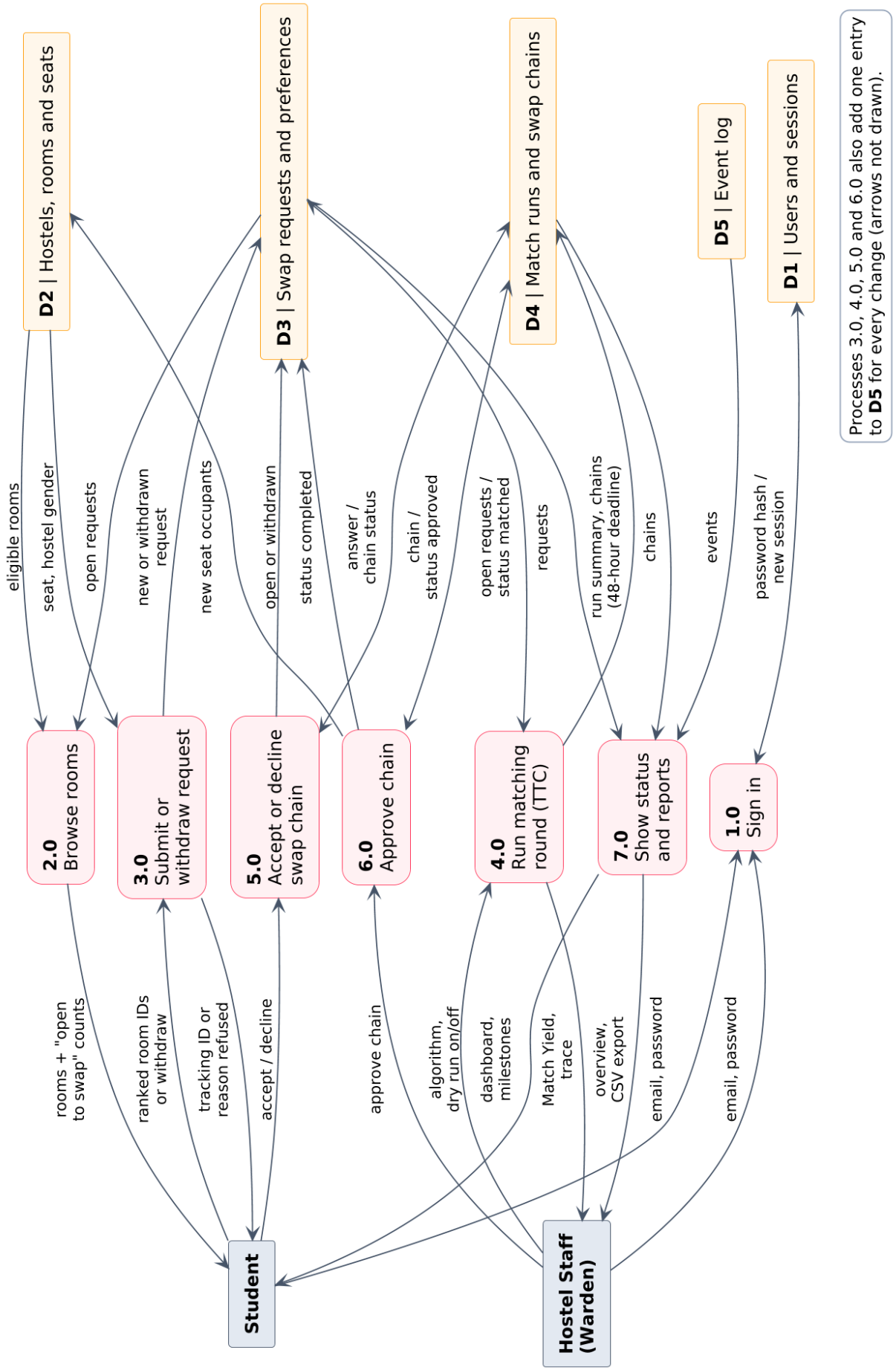


Figure 2.3.: Data flow diagram, Level 1 (Gane–Sarson notation). Square boxes are people, rounded boxes are processes, and boxes with a D number are data stores. Processes 3.0, 4.0, 5.0 and 6.0 also add one line to D5 (Event log) for every change; those four arrows are left out so the picture stays readable.

- One hostel office uses the system at a time. SQLite allows only one writer at a time, which is enough for this.
- The project must fit in one semester (2026–27 odd semester).

2.6. Design

This section describes what each kind of user does, step by step, and how a request and a swap chain change status over time.

2.6.1. Student activities

1. Sign in with a Thapar email address and password.
2. Browse rooms in the hostels they may live in, using filters.
3. Rank up to five rooms, best first, and submit. The system checks the list and either saves it with a tracking ID or explains what is wrong.
4. Wait in the *pool* until the warden runs a round. While the request is still open, the student may withdraw it.
5. If the round places them in a chain, review the whole chain on the dashboard and accept or decline before the 48-hour deadline.
6. After everyone accepts and the warden approves, see the new room on the dashboard.

2.6.2. Hostel staff (warden) activities

1. Sign in on the *Hostel Staff* tab.
2. See the overview: requests in the pool, chains waiting for answers, chains waiting for approval, and the last round's results.
3. Optionally compare TTC with direct one-to-one swaps on the current pool, or preview a round without saving it (dry run).
4. Run a TTC matching round, and read the step-by-step trace that explains every chain found.
5. Approve each chain that every member has confirmed.
6. Export all requests to a spreadsheet (CSV) for the hostel office.

Figure 2.4 shows both sets of activities together, in swimlanes, for the life of one request.

2.6.3. How a request and a chain change status

A swap request is always in one of four states (Figure 2.5). A swap chain, also called a *cycle*, is always in one of four states (Figure 2.6). The two are linked: when a chain is dissolved, each member's request goes back to *open* or becomes *withdrawn*, depending on how that member answered.

2.6.4. How the parts work together for one swap

Figure 2.7 follows one swap from the student's request to the warden's approval, through the real layers of the system: the page, the server's routes, the services, the matching engine and the database.

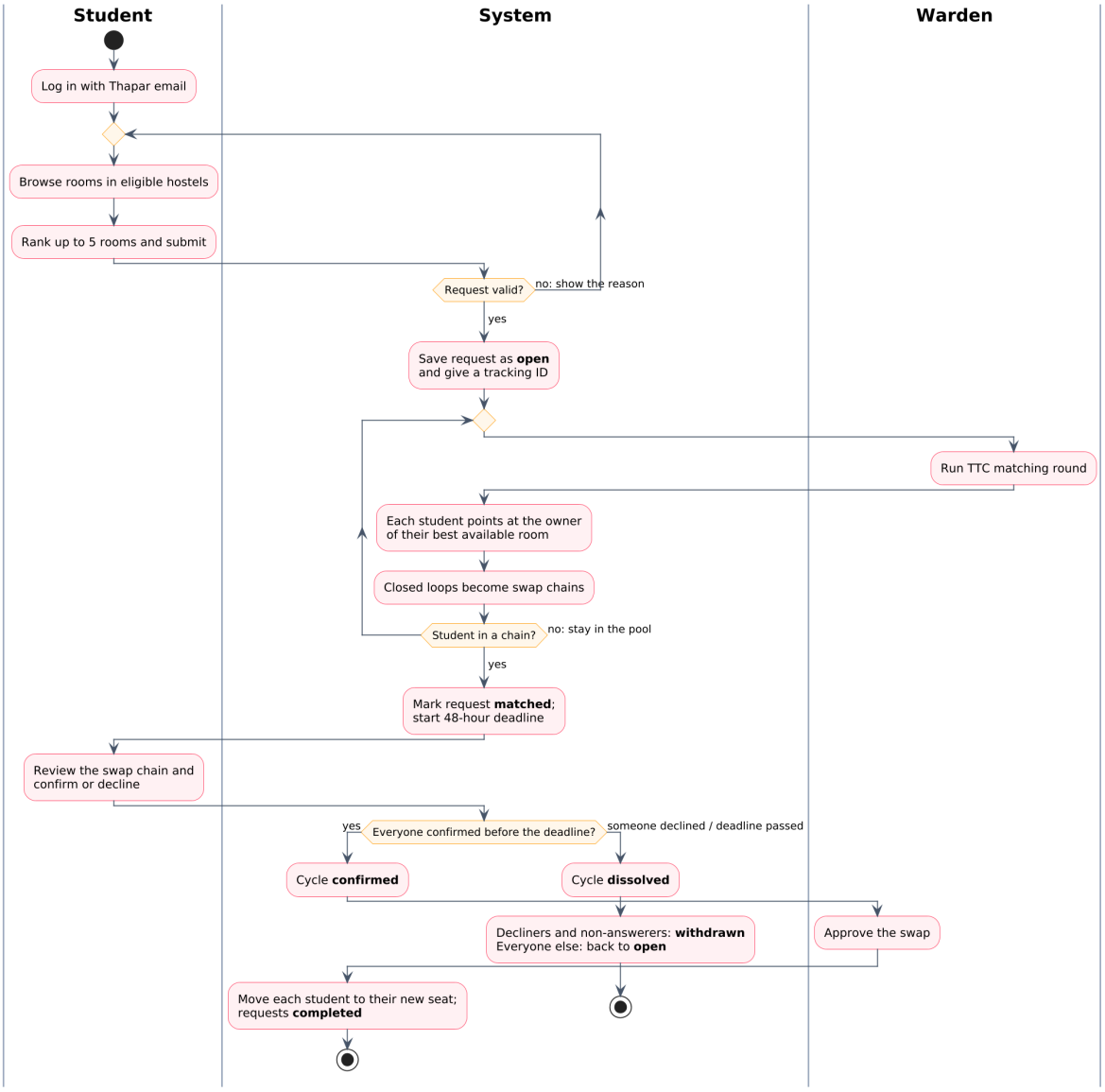


Figure 2.4.: Activity diagram: the life of one swap request, with a swimlane for the student, the system and the warden.

2.7. User Documentation

- A documentation website (built with MkDocs from the `docs/` folder) covers getting started, architecture, data model, API, matching engine, frontend, testing, diagrams and known limits.
- The running server shows an interactive API page at `/docs`, where each call can be tried.
- A study guide (`docs/study/`) explains each part in plain language for the four team members.
- The sign-in page lists the demo accounts, so a demonstration cannot stall on a forgotten password.

2.8. Assumptions and Dependencies

- Every student who submits a request already occupies exactly one seat, and the hostel and room data are correct.

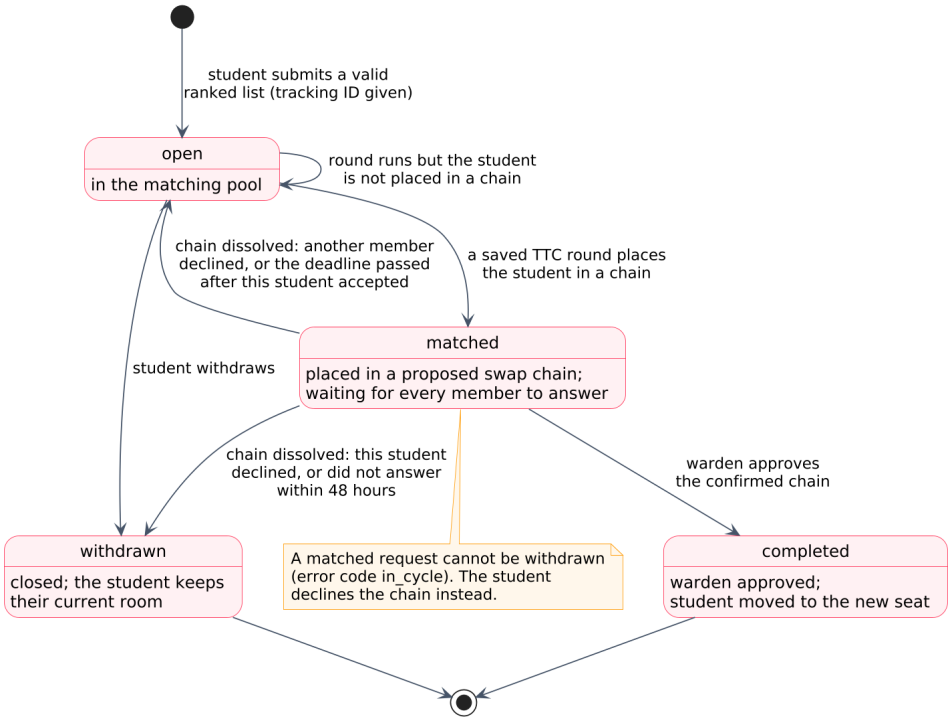


Figure 2.5.: State diagram: the status of one swap request.

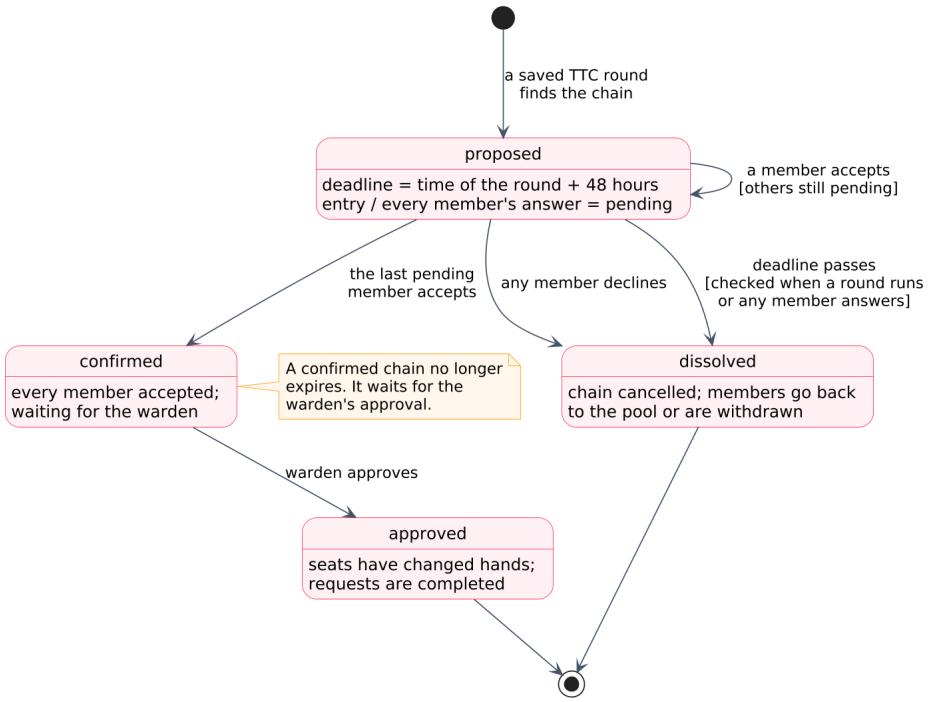


Figure 2.6.: State diagram: the status of one swap chain (cycle).

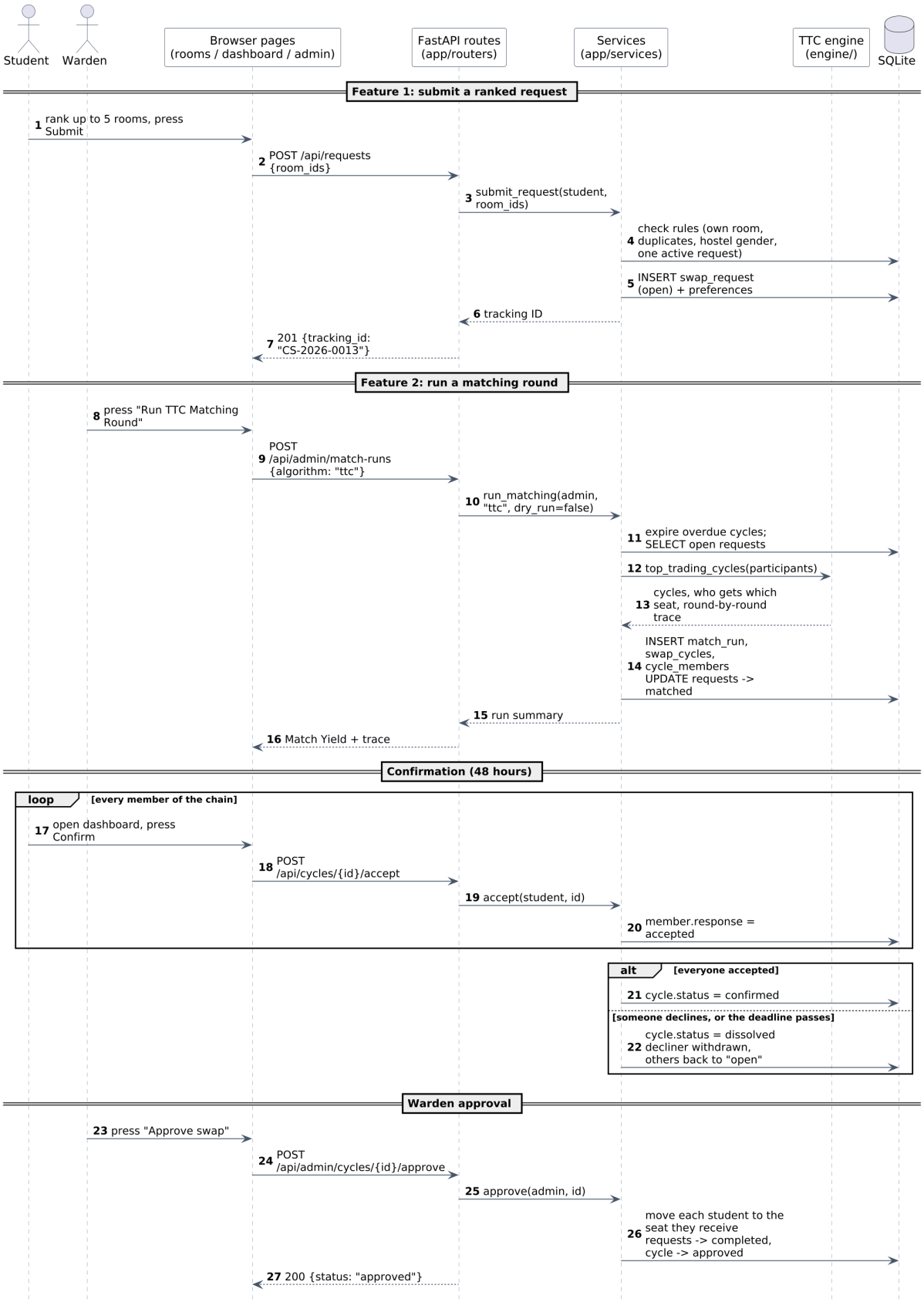


Figure 2.7.: Sequence diagram: one swap from request to approval. The *loop* box repeats for every member of the chain; the *alt* box shows the two ways the confirmation step can end.

- Each hostel houses one gender, and a student may only move within hostels of their own gender.
- The warden runs matching rounds regularly (for example, weekly).
- Until notifications are built (`FR-CONF-06`), students check their dashboard within 48 hours of a round.
- The hostel office keeps its own master records and copies approved swaps into them.
- Only development needs the internet: Python packages, GitHub (code and automated tests) and the PlantUML server (diagrams).

3. External Interface Requirements

3.1. User Interfaces

CampusSwap has four pages. The screenshots below come from the working prototype, running on its made-up demo data after one real matching round.

Rules that every page follows:

- While data loads, the page shows grey placeholder shapes (a *loading skeleton*) instead of an empty screen.
- Every error appears as a short, plain message: a pop-up with the server's explanation, or a box on the page with a *Try again* button.
- Empty states explain what to do next (for example, "No request yet" with a link to Explore Rooms).
- Long names are shortened with "…" and shown in full on hover.
- Layouts change for phone screens; nothing needs sideways scrolling.

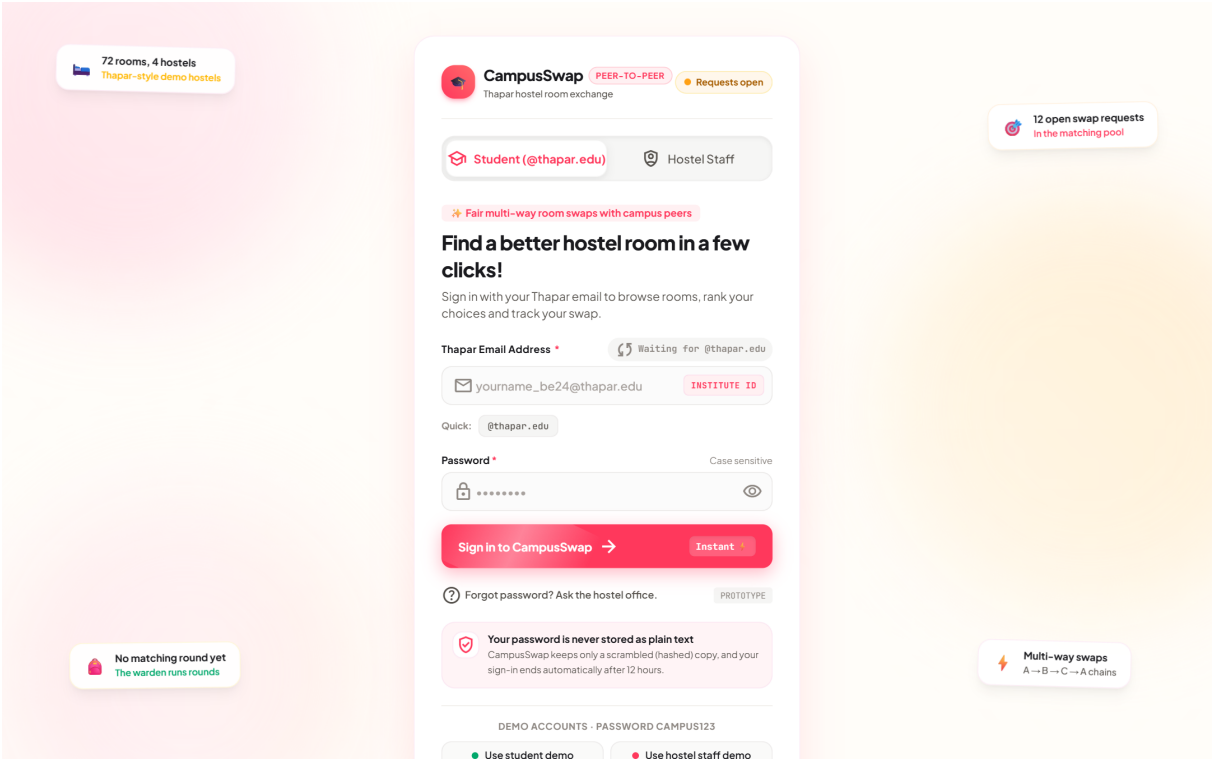


Figure 3.1.: Sign-in page. A switch chooses Student or Hostel Staff. The floating cards show live public numbers from the server.

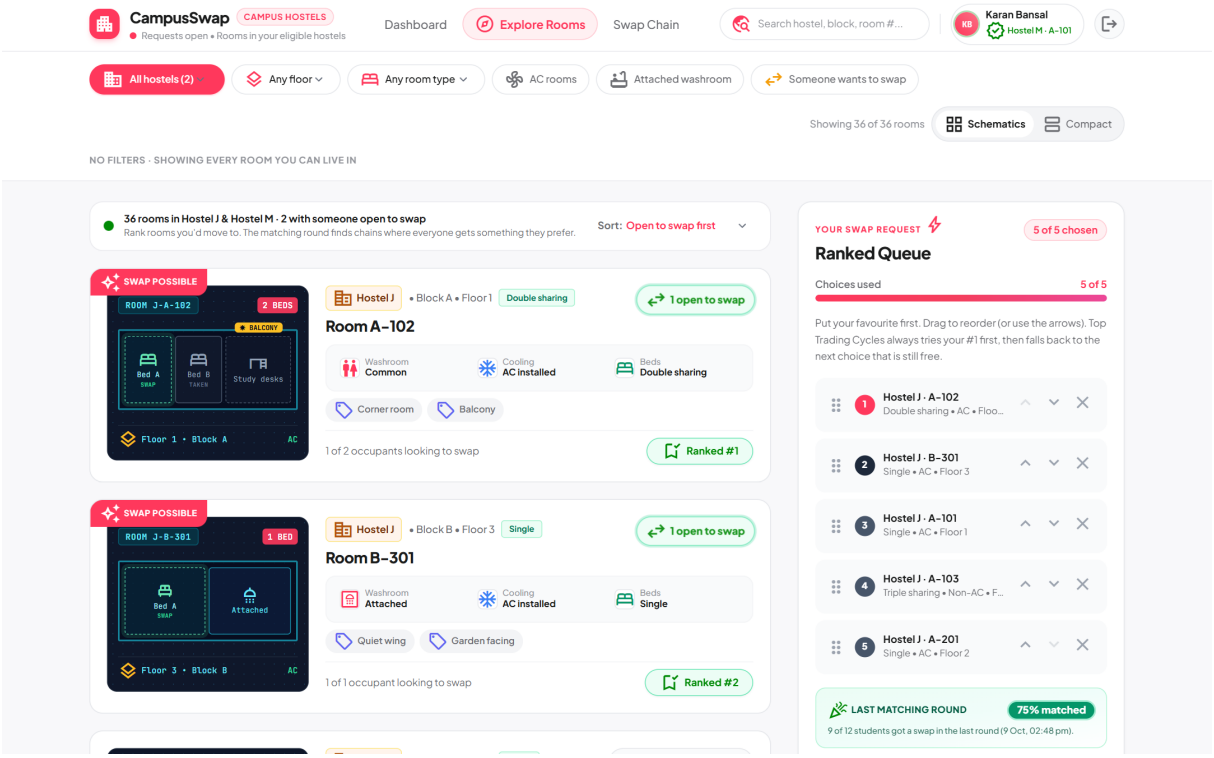


Figure 3.2.: Explore Rooms page. Filters at the top; each room card is drawn from the room's data and shows how many occupants are open to a swap. The ranked list on the right holds up to five choices and can be reordered.

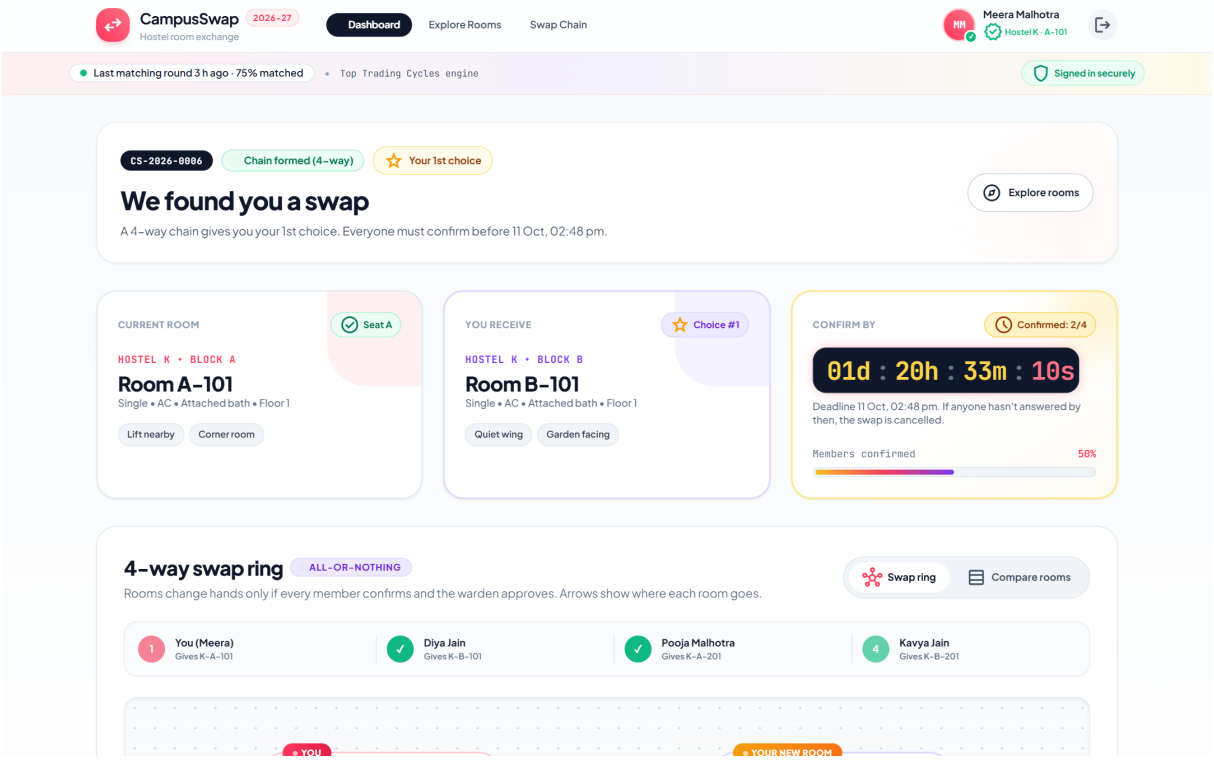


Figure 3.3.: Student dashboard after a round: the tracking ID, current room, the room on offer, and a countdown to the 48-hour deadline with how many members have confirmed.

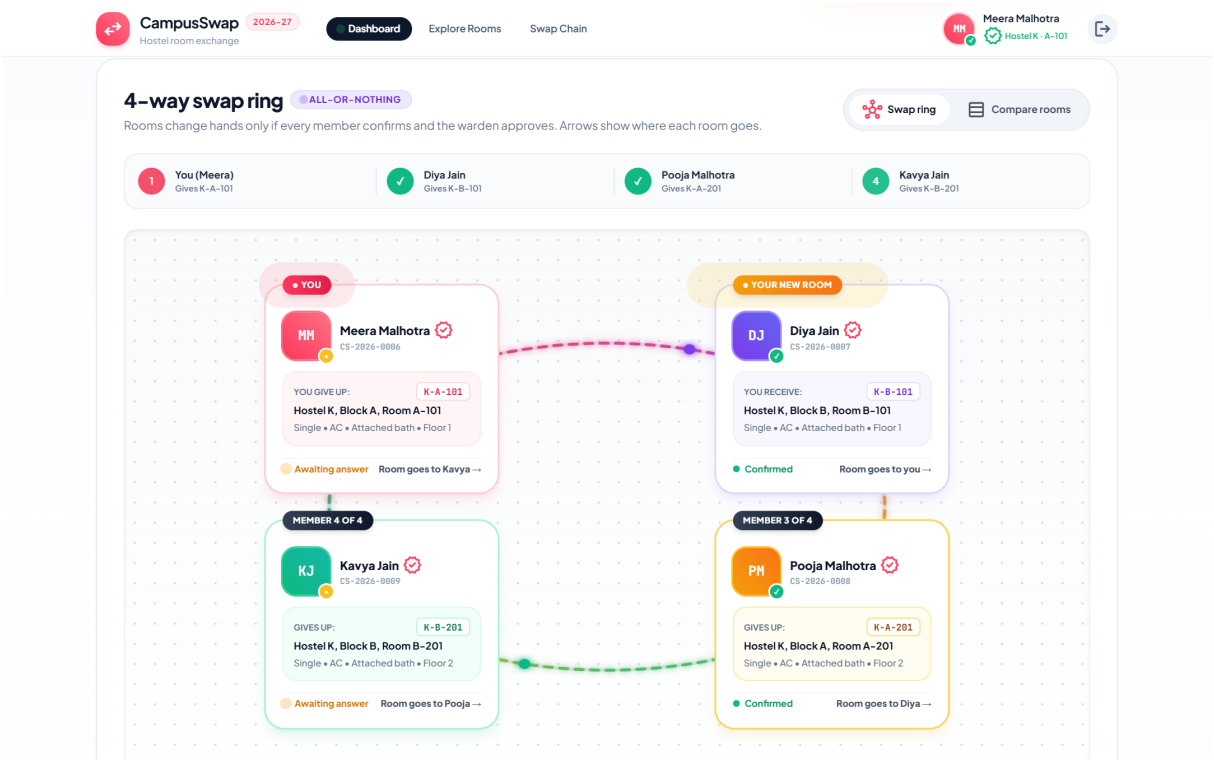
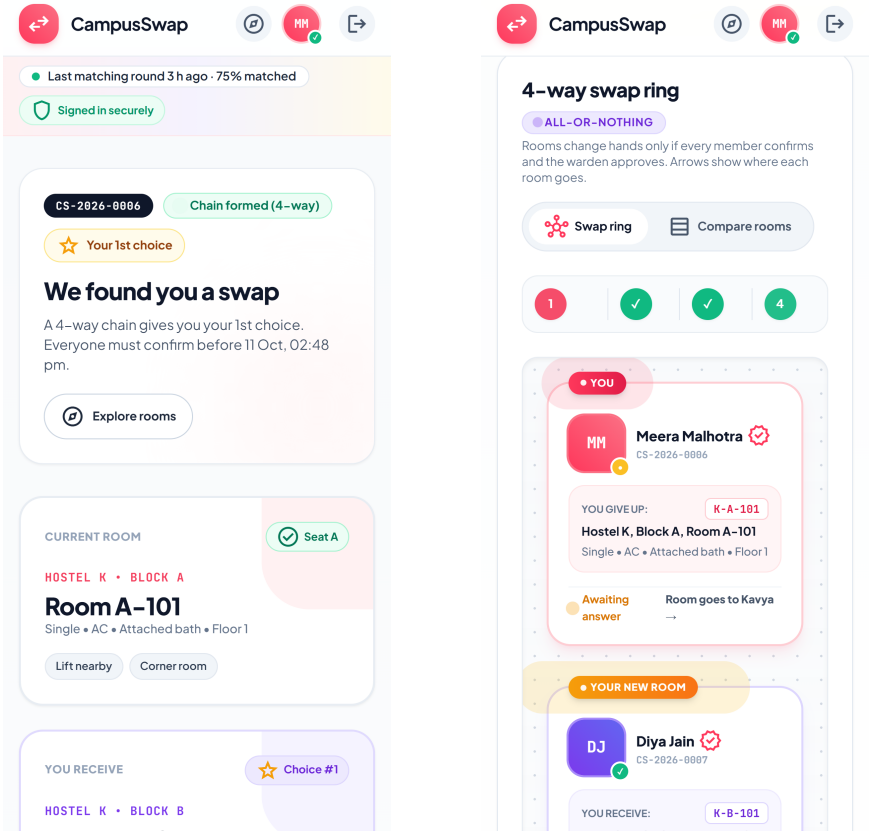


Figure 3.4.: The swap ring for a real 4-way chain found by the round. Arrows show where each room goes; each card shows whether that member has confirmed.



(a) Top of the dashboard

(b) The swap ring as stacked cards

Figure 3.5.: The same dashboard on a phone screen, 390 pixels wide.

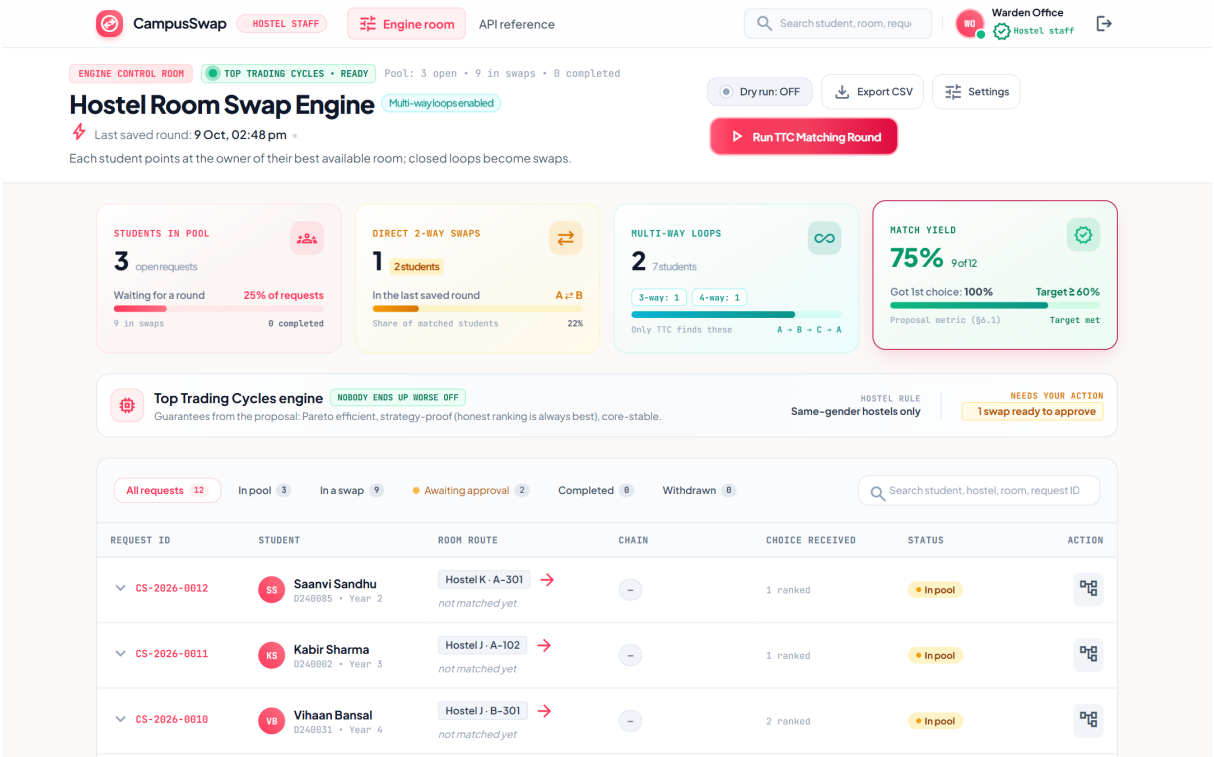


Figure 3.6.: Hostel staff page (“Engine room”): run a round, see the Match Yield against the 60% target, the count of 2-way swaps and multi-way loops, and every request with its chain and status.

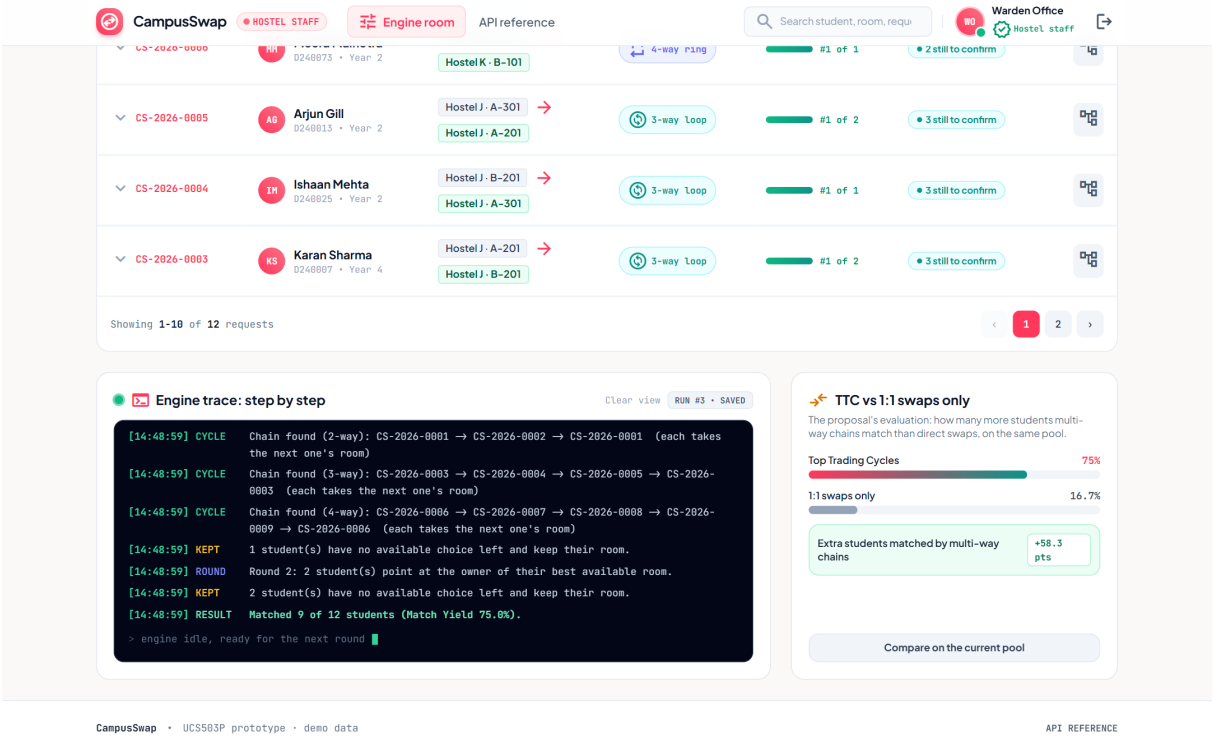


Figure 3.7.: Lower part of the hostel staff page: the engine’s real step-by-step trace for the round, and the comparison on the same pool (TTC matched 75% of students, direct one-to-one swaps only 16.7%).

3.2. Hardware Interfaces

CampusSwap needs no special hardware. Users need a device with a web browser. The server can be an ordinary laptop or a small cloud server; the whole database is one file.

3.3. Software Interfaces

3.3.1. Web API

The pages talk to the server only through the JSON interface below (an *API*). All paths start with `/api`.

Table 3.1.: Web API

Method	Path	Who	What it does
POST	`/auth/login`	anyone	Body `{email, password, role}`. Sets the session cookie.
POST	`/auth/logout`	anyone	Ends the session.
GET	`/auth/me`	signed in	The current user.
GET	`/auth/status`	anyone	The current user or `null`; never an error.
GET	`/public/stats`	anyone	Open requests, rooms, hostels, requests per day for 7 days, last round. No personal data.
GET	`/rooms`	student	Rooms in eligible hostels, with `is_own` and `open_to_swap`.
GET	`/me/dashboard`	student	Room, request, chain, milestones and activity.
POST	`/requests`	student	Body `{room_ids}`, 1–5 rooms, best first. Returns `201` and the tracking ID.
DELETE	`/requests/current`	student	Withdraw the open request.
POST	`/cycles/{id}/accept`	student	Accept a place in a proposed chain.
POST	`/cycles/{id}/decline`	student	Decline; the chain is cancelled for everyone.
GET	`/admin/overview`	staff	Counts, latest round, latest comparison, latest trace.
GET	`/admin/requests`	staff	Every request with its chain.
POST	`/admin/match-runs`	staff	Body `{algorithm, dry_run}`. Runs a round.
POST	`/admin/compare`	staff	Dry-runs TTC and one-to-one swaps on the same pool.
POST	`/admin/cycles/{id}/approve`	staff	Approve a confirmed chain; seats change hands.

Method	Path	Who	What it does
GET	`/admin/export.csv`	staff	Download all requests as `campusswap-requests.csv`.

Every error has the same shape, so a page can always show a clear message:

```
{"error": "own_room", "message": "You can't rank the room you already live in."}
```

Table 3.2.: Error responses

Status	Meaning	Error codes
400	The request breaks a rule	`empty_list`, `too_many`, `duplicate_room`, `own_room`, `unknown_room`, `wrong_hostel_gender`, `unknown_algorithm`, `pairwise_must_be_dry_run`
401	Not signed in, or wrong password	`not_logged_in`, `bad_credentials`
403	Signed in, but not allowed	`forbidden`, `wrong_role`, `not_member`
404	Does not exist	`no_request`, `no_cycle`
409	Not possible right now	`already_active`, `in_cycle`, `no_seat`, `cycle_closed`, `already_answered`, `not_confirmed`, `run_in_progress`
422	The body has the wrong shape	`invalid_input`

3.3.2. Internal interfaces

- **Server and database:** SQLAlchemy 2.0 over SQLite, with foreign keys switched on. Times are stored in UTC.
- **Server and matching engine:** plain Python function calls. The server passes a list of participants (each with an ID, the seat they own and the seats they want, best first) to `top_trading_cycles()` and receives the chains, who gets which seat, who keeps their seat, and a round-by-round record.

3.4. Communications Interfaces

- Browser and server communicate over HTTP, with JSON bodies in UTF-8.
- After sign-in the browser holds a session cookie, `cs_session`. It is marked *HttpOnly* (page scripts cannot read it) and *SameSite=Lax* (other websites cannot send it in form posts), and expires after 12 hours.
- For a real deployment the server shall be reached only over HTTPS, with the cookie also marked *Secure* (`NFR-SEC-07`, **Planned**).
- Phase 2 adds email notifications (`FR-CONF-06`, **Planned**).

4. System Features

CampusSwap is a room-swap system. Its core is collecting ranked requests and finding fair chains of swaps; everything else supports that.

4.1. Description and Priority

The features, from most to least important:

1. **Run a matching round** (High). The reason the system exists: it finds the swap chains.
2. **Submit and withdraw a swap request** (High). Without requests there is nothing to match.
3. **Accept or decline a swap chain** (High). Nobody is moved without agreeing.
4. **Approve a chain** (High). The warden's final check before rooms change.
5. **Sign in and sessions** (High). Keeps each user to their own actions.
6. **Browse rooms** (Medium). Helps students choose; a student could rank rooms without it.
7. **Status tracking and reports** (Medium). Builds trust and gives the hostel office its records.
8. **Faculty slot exchange** (Low, Phase 2). Reuses the same engine for lecture slots.

The sequence diagram in Section 2.6 (Figure 2.7) shows how the main features work together for one swap, from the student's request to the warden's approval.

4.2. Functional Requirements

4.2.1. Sign in and sessions

Description and priority. Users sign in with an email address, a password and a role (Student or Hostel Staff). Priority: High.

Stimulus and response.

1. The user enters an email and password and presses *Sign in*. The system checks them and opens the right home page, or shows "Wrong email or password."
2. The user presses *Sign out*. The system ends the session and returns to the sign-in page.

Table 4.1.: Requirements: sign in and sessions

ID	Requirement	Priority	Status
`FR-AUTH-01`	The system shall let a user sign in with an email address, a password and a role (Student or Hostel Staff).	High	**Built**
`FR-AUTH-02`	The system shall ignore letter case and surrounding spaces in the email address.	Medium	**Built**

ID	Requirement	Priority	Status
FR-AUTH-03	For an unknown email or a wrong password, the system shall give the same answer (`401 bad_credentials`) and take about the same time, so nobody can find out which accounts exist.	High	**Built**
FR-AUTH-04	If the password is right but the user chose the wrong role tab, the system shall say which tab to use (`403 wrong_role`).	Medium	**Built**
FR-AUTH-05	On sign-in the system shall start a session that lasts 12 hours, held in the `cs_session` cookie.	High	**Built**
FR-AUTH-06	Signing out shall delete the session on the server and clear the cookie.	High	**Built**
FR-AUTH-07	Every student action shall require a signed-in student, and every staff action a signed-in staff member (otherwise `401 not_logged_in` or `403 forbidden`).	High	**Built**
FR-AUTH-08	The sign-in page shall accept only addresses ending in `@thapar.edu`.	Medium	**Built**
FR-AUTH-09	The server shall also refuse sign-in for addresses outside `@thapar.edu` (today only the page checks this).	Medium	**Planned**
FR-AUTH-10	The system shall slow down or block repeated failed sign-in attempts.	Medium	**Planned**
FR-AUTH-11	Students shall be able to sign in with the institute's Microsoft account (single sign-on).	Low	**Planned**

4.2.2. Browse rooms

Description and priority. A student sees every room they could move to and picks the ones to rank. Priority: Medium.

Stimulus and response.

1. The student opens *Explore Rooms*. The system lists every room in the hostels of the student's gender.
2. The student changes a filter or types a search. The list updates at once.
3. The student presses *Rank in Preferences* on a room. The room joins the ranked list on the right.

Table 4.2.: Requirements: browse rooms

ID	Requirement	Priority	Status
FR-ROOM-01	The system shall show a student only the rooms in hostels that house the student's gender.	High	**Built**
FR-ROOM-02	For each room the system shall show the hostel, block, floor, room number, number of beds (1–3), air conditioning, washroom type (attached or common) and tags.	High	**Built**

ID	Requirement	Priority	Status
FR-ROOM-03	For each room the system shall show how many of its other occupants have an open request (“open to swap”), without showing their names.	Medium	**Built**
FR-ROOM-04	The system shall mark the student’s own room and not let it be ranked.	High	**Built**
FR-ROOM-05	The page shall filter by hostel, floor, number of beds, air conditioning, attached washroom and “someone wants to swap”, and search by hostel, block or room number.	Medium	**Built**
FR-ROOM-06	The page shall offer a detailed view and a compact view, split into pages.	Low	**Built**
FR-ROOM-07	A student shall be able to rank a room type (for example “any single AC room on floor 1”) instead of one specific room.	Low	**Planned**

4.2.3. Submit and withdraw a swap request

Description and priority. A student turns their ranked list into a swap request, or takes it back. Priority: High.

Stimulus and response.

1. The student presses *Submit*. If the list breaks a rule, the system shows the reason and saves nothing. Otherwise it saves the request as *open* and shows its tracking ID.
2. The student presses *Withdraw*. If the request is still open, the system marks it *withdrawn*.

Table 4.3.: Requirements: submit and withdraw a swap request

ID	Requirement	Priority	Status
FR-REQ-01	A student shall be able to rank one to five rooms, best first, and submit them as one request.	High	**Built**
FR-REQ-02	Before submitting, the student shall be able to reorder the list by dragging (desktop) or with up and down buttons (any screen), and remove rooms.	Medium	**Built**
FR-REQ-03	The system shall refuse an empty list (`empty_list`), more than five rooms (`too_many`) and the same room twice (`duplicate_room`).	High	**Built**
FR-REQ-04	The system shall refuse a request from a student who has no seat (`no_seat`) or who already has an open or matched request (`already_active`).	High	**Built**
FR-REQ-05	The system shall refuse a room that does not exist (`unknown_room`), the student’s own room (`own_room`) and a room in a hostel for the other gender (`wrong_hostel_gender`).	High	**Built**

ID	Requirement	Priority	Status
FR-REQ-06	A valid request shall be saved as *open*, together with its ranked rooms and the seat the student offers, and given a tracking ID of the form CS-YYYY-NNNN (year, then a four-digit number).	High	**Built**
FR-REQ-07	A student shall be able to withdraw a request only while it is open. A matched request cannot be withdrawn (in_cycle); the student declines the chain instead.	High	**Built**
FR-REQ-08	Every submit and withdraw shall be recorded in the event log.	Medium	**Built**

4.2.4. Run a matching round

Description and priority. Hostel staff run Top Trading Cycles over every open request and save the chains it finds. Priority: High.

How Top Trading Cycles works. Every student in the pool owns a seat and wants some rooms, best first. Repeat until nobody is left:

1. Each remaining student points at the owner of their best room that is still available. A student with no choice left points at themselves.
2. Every closed loop of pointers is a chain: each member takes the seat of the person they point at. A student pointing at themselves keeps their room.
3. Everyone in a loop is removed, together with their seats.

Because every student points at exactly one person, at least one loop always exists, so the method always finishes. TTC is known to give an outcome that no other assignment can improve for someone without hurting someone else, in which nobody gains by ranking dishonestly, and from which no group can break away and do better among themselves [3, 5].

Stimulus and response.

1. Staff press *Run TTC Matching Round*. The system dissolves overdue chains, runs TTC on the pool, saves the chains, and shows the Match Yield and the step-by-step trace.
2. Staff press *Compare on the current pool*. The system runs TTC and direct one-to-one swaps as dry runs and shows both results side by side.

Table 4.4.: Requirements: run a matching round

ID	Requirement	Priority	Status
FR-MATCH-01	Hostel staff shall be able to run a TTC round over every open request (the pool).	High	**Built**
FR-MATCH-02	Before building the pool, the system shall dissolve every proposed chain whose deadline has passed.	High	**Built**
FR-MATCH-03	In a room with several beds, the system shall offer the beds of pool members in a fixed order (lowest seat number first), so the same pool always gives the same result.	High	**Built**

ID	Requirement	Priority	Status
FR-MATCH-04	The system shall never assign a student a room they did not rank; a student with no possible swap keeps their room.	High	**Built**
FR-MATCH-05	A saved round shall turn each loop of two or more students into a *proposed* chain with a deadline 48 hours after the round, and mark their requests *matched*.	High	**Built**
FR-MATCH-06	A dry run shall compute and record the round's numbers but create no chains and change no request.	Medium	**Built**
FR-MATCH-07	For each round the system shall report the pool size, the number matched, the Match Yield (matched ÷ pool × 100, one decimal place), the share of matched students who got their first choice, the number of chains of each length, and a plain-language step-by-step trace.	High	**Built**
FR-MATCH-08	Only one round shall run at a time; a second attempt gets `409 run_in_progress`.	High	**Built**
FR-MATCH-09	The system shall compare TTC with direct one-to-one swaps on the same pool at the same moment. One-to-one swaps shall only ever run as a dry run (`pairwise_must_be_dry_run`).	Medium	**Built**
FR-MATCH-10	Rounds shall run automatically on a schedule set by the hostel office.	Low	**Planned**

4.2.5. Accept or decline a swap chain

Description and priority. Every member of a proposed chain must agree before anyone moves. One "no" cancels the whole chain. Priority: High.

Stimulus and response.

1. A member presses *Confirm*. The system records the answer. When the last member confirms, the chain becomes *confirmed* and waits for the warden.
2. A member presses *Decline*. The chain is dissolved at once. The decliner's request is withdrawn; everyone else goes back to the pool for the next round.
3. The deadline passes. At the next round or answer, the chain is dissolved. Members who did not answer are withdrawn; members who accepted go back to the pool.

Table 4.5.: Requirements: accept or decline a swap chain

ID	Requirement	Priority	Status
FR-CONF-01	The system shall show each member the whole chain: who gives which room, who has answered, which choice the member receives, and the time left before the deadline.	High	**Built**
FR-CONF-02	A member shall be able to accept once. When every member has accepted, the chain shall become *confirmed*.	High	**Built**

ID	Requirement	Priority	Status
FR-CONF-03	A member shall be able to decline. The chain shall be dissolved, the decliner's request withdrawn, and every other member's request put back to *open*.	High	**Built**
FR-CONF-04	A proposed chain whose deadline has passed shall be dissolved when the next round runs or any member answers. Members who did not answer are withdrawn; members who accepted go back to *open*.	High	**Built**
FR-CONF-05	The system shall refuse an answer from a non-member (`not_member`), for a chain that is no longer proposed (`cycle_closed`), or from a member who already answered (`already_answered`).	High	**Built**
FR-CONF-06	The system shall email each member when a chain is found and again before the deadline.	Medium	**Planned**

4.2.6. Approve a chain

Description and priority. The warden gives the final approval; only then do rooms change. Priority: High.

Stimulus and response. The warden presses *Approve* on a confirmed chain. The system moves each member to the seat they receive and marks their requests *completed*.

Table 4.6.: Requirements: approve a chain

ID	Requirement	Priority	Status
FR-APPR-01	Hostel staff shall be able to approve only a chain in which every member has accepted (otherwise `409 not_confirmed`).	High	**Built**
FR-APPR-02	On approval, in one database transaction, every member shall move to the seat they receive, their requests become *completed*, and the chain becomes *approved*.	High	**Built**
FR-APPR-03	The system shall produce a signed-off approval form (PDF) listing every member, for the hostel office.	Medium	**Planned**

4.2.7. Status tracking and reports

Description and priority. Students always know where their request stands; staff always have a complete record. Priority: Medium.

Stimulus and response. A student opens the dashboard and sees their room, request and chain. Staff open the engine room and see every request; pressing *Export CSV* downloads them as a spreadsheet.

Table 4.7.: Requirements: status tracking and reports

ID	Requirement	Priority	Status
FR-TRACK-01	The student dashboard shall show the current room, the active request and its tracking ID, the chain (if any), and five milestones: Request submitted, In the matching pool, Match found, Everyone confirmed, Warden approved.	High	**Built**
FR-TRACK-02	The dashboard shall list the last 20 events about the student, newest first.	Medium	**Built**
FR-TRACK-03	The dashboard shall explain what to do next when the student has no request, is waiting in the pool, or was not matched.	Medium	**Built**
FR-TRACK-04	The staff overview shall show the number of open, matched and completed requests, the chains waiting for approval, the latest saved round, the latest comparison, the latest trace, and the Match Yield of the last ten saved rounds.	High	**Built**
FR-TRACK-05	The staff request table shall show every request with the student's name, roll number and year, the room offered, the ranked rooms, the room assigned, the choice received, and the chain with each member's answer; it shall be searchable and filterable by status.	High	**Built**
FR-TRACK-06	Staff shall be able to download all requests as `campusswap-requests.csv` with the columns `tracking_id`, `student`, `roll_no`, `status`, `offered_room`, `assigned_room`, `received_rank`, `cycle_id`, `cycle_length`, `cycle_status`, `submitted_at`.	Medium	**Built**
FR-TRACK-07	The sign-in page shall show public numbers (open requests, rooms, hostels, requests per day for the last seven days, last round) without any personal data.	Low	**Built**
FR-TRACK-08	Staff shall be able to download the full event log as an audit trail.	Medium	**Planned**

4.2.8. Faculty slot exchange (Phase 2)

Description and priority. Faculty members swap lecture slots using the same engine: each owns a slot and ranks slots they prefer. Priority: Low.

Table 4.8.: Requirements: faculty slot exchange

ID	Requirement	Priority	Status
FR-SLOT-01	Faculty shall be able to rank lecture slots they would prefer to their own, and the matching engine shall find slot-swap chains in the same way as for rooms.	Low	**Planned**

5. Other Nonfunctional Requirements

5.1. Performance Requirements

Table 5.1.: Performance requirements

ID	Requirement	Priority	Status
NFR-PERF-01	The matching step for a pool of 2,000 requests shall take less than 2 seconds. An automated test checks this; on the team's laptop it took 17 ms (median of five runs).	High	**Built**
NFR-PERF-02	The time for a round shall grow at most with the square of the pool size: each student's pointer only moves forward through their own list.	Medium	**Built**
NFR-PERF-03	The pages shall be usable on a phone screen 390 pixels wide without sideways scrolling.	High	**Built**
NFR-PERF-04	The pages shall load and work with no internet access (fonts, icons and styles are stored locally).	Medium	**Built**

5.2. Safety Requirements

Here, safety means protecting data from being lost, mixed up or exposed.

Table 5.2.: Safety requirements

ID	Requirement	Priority	Status
NFR-SAFE-01	Every change shall happen inside one database transaction: a matching round or an approval is saved completely or not at all.	High	**Built**
NFR-SAFE-02	The database shall make it impossible for two students to occupy the same seat (a unique rule on the seat's occupant).	High	**Built**
NFR-SAFE-03	The database shall reject any row that points at something that does not exist (foreign keys switched on).	High	**Built**
NFR-SAFE-04	A request shall not rank the same room twice or use the same rank twice (unique rules in the database).	High	**Built**
NFR-SAFE-05	The prototype shall contain only made-up data.	High	**Built**
NFR-SAFE-06	Students' names shall not be shown while browsing rooms; they are shown only to members of the same chain and to hostel staff.	High	**Built**

5.3. Security Requirements

Table 5.3.: Security requirements

ID	Requirement	Priority	Status
NFR-SEC-01	Passwords shall never be stored as plain text. The system stores a salted *scrypt* hash ($N = 2^{14}$, $r = 8$, $p = 1$, 16-byte random salt) and compares hashes in constant time.	High	**Built**
NFR-SEC-02	A session token shall be 32 random bytes. The server shall store only its SHA-256 hash, so a leaked database cannot be used to sign in.	High	**Built**
NFR-SEC-03	The session cookie shall be HttpOnly and SameSite=Lax, and expire after 12 hours.	High	**Built**
NFR-SEC-04	The server shall check the user's role on every request; hiding a button on the page is never the only protection.	High	**Built**
NFR-SEC-05	All text that came from users shall be escaped before it is put on a page, so nobody can inject their own script.	High	**Built**
NFR-SEC-06	Error messages shall not reveal whether an email address has an account.	Medium	**Built**
NFR-SEC-07	A deployed system shall be reached only over HTTPS, with the cookie marked Secure.	High	**Planned**
NFR-SEC-08	Repeated failed sign-ins shall be limited (see FR-AUTH-10).	Medium	**Planned**

5.4. Software Quality Attributes

- **Correctness and testability.** 89 automated tests check the engine, every business rule and the web API (Appendix C). The engine is tested with made-up participants and no database, including 50 random pools of 40 students in which nobody is ever given a seat they did not ask for.
- **Reliability.** GitHub Actions runs every test on each push and pull request, so a change that breaks a rule is caught before it is merged.
- **Maintainability.** Each rule lives in one place (the services layer). Pages never repeat a rule; they only show the server's answer.
- **Portability.** The server runs on Windows, Linux and macOS. Moving from SQLite to PostgreSQL needs only a different database address.
- **Usability.** Plain-language messages, a demo-accounts hint on the sign-in page, and the loading, empty and error states described in Section 3.1.
- **Explainability.** Every round produces a plain-language trace ("Chain found (3-way): ..."), so staff and students can see why a chain was proposed.

5.5. Business Rules

Table 5.4.: Business rules

ID	Rule
BR-01	A student may only rank rooms in hostels that house their own gender.
BR-02	A request ranks one to five different rooms, never the student's own room.
BR-03	A student has at most one active (open or matched) request at a time.
BR-04	Only an open request may be withdrawn.
BR-05	Only hostel staff may run rounds, compare methods, approve chains and export records.
BR-06	Every member of a chain must accept within 48 hours of the round.
BR-07	A chain is all-or-nothing: one decline cancels it for everyone.
BR-08	Rooms change only after the warden approves a confirmed chain.
BR-09	Nobody is ever moved to a room they did not rank; a student with no swap keeps their room.
BR-10	CampusSwap's approval is a record for the hostel office, which still updates its own master allotment records.
BR-11	A one-to-one comparison never changes any request or room.

6. Other Requirements

6.1. Maintenance

CampusSwap will be used for many semesters, so it must stay easy to change.

- Every new rule shall come with an automated test, and the full test suite shall pass before a change is merged.
- Diagrams are kept as text (PlantUML) next to the code, and redrawn with `python diagrams/render.py`, so they can be updated with the code.
- The matching engine shall stay free of database and web code, so it can be reused (for example, for faculty slot exchange).

6.2. Data and privacy

- The system shall collect only what matching needs: name, roll number, gender, year, branch, current seat and ranked rooms.
- Before a pilot with real students (planned with 20 to 40 volunteers), each participant shall give consent, and the data shall be deleted after the pilot.
- Sessions expire after 12 hours; expired sessions can be removed at any time without losing any other data.

6.3. Deployment (Phase 2)

- The system shall run on a server reachable only over HTTPS (`NFR-SEC-07`).
- For many users at once, the database shall move to PostgreSQL.
- Long rounds, notifications and deadline checks may move to a background worker if pools become very large.

A. Glossary

Term	Meaning
API	Application Programming Interface: the fixed set of requests the pages can send to the server.
Chain (cycle)	A closed loop of students in which each takes the next one's seat. A 2-way chain is a direct swap.
Core-stable	No group of students can leave the result and swap among themselves so that all of them do better.
CSV	Comma-separated values: a plain spreadsheet file.
Data flow diagram (DFD)	A picture of where data comes from, which process changes it and where it is stored.
Dry run	A round that is calculated and reported but changes nothing.
ER diagram	Entity–relationship diagram: the database tables and how they link.
Hash	A one-way scrambled form of a secret. The original cannot be worked out from it.
HTTP / HTTPS	The language browsers and servers use to talk. HTTPS is the encrypted version.
HttpOnly cookie	A cookie that page scripts cannot read, so injected code cannot steal it.
JSON	A simple text format for sending structured data.
Match Yield	Students placed in a chain ÷ students in the pool × 100.
Pareto efficient	No other assignment makes someone happier without making someone else unhappier.
Pool	All open requests at the moment a round runs.
Scrypt	A deliberately slow password-hashing method, so stolen hashes are hard to crack.
Seat	One bed in a room. A double room has two seats, each with its own occupant.
Session	The server's memory that a browser has signed in.
SHA-256	A standard hash function.
Single sign-on (SSO)	Signing in with an existing institute account instead of a separate password.
SQLite	A database stored in a single file.
Strategy-proof	Ranking rooms honestly is always a student's best move.
Top Trading Cycles (TTC)	The matching method CampusSwap uses (Shapley and Scarf, 1974).
Tracking ID	The request's public ID, for example `CS-2026-0013`.
Warden	Hostel staff member who runs rounds and approves swaps.

B. Analysis Models

The diagrams already shown in this document are the context diagram (Figure 1.1), architecture (Figure 2.1), use case diagram (Figure 2.2), Level 1 data flow diagram (Figure 2.3), activity diagram (Figure 2.4), state diagrams (Figures 2.5 and 2.6) and sequence diagram (Figure 2.7). This appendix adds the class diagram, drawn in two parts so that each stays readable, and the ER diagram.

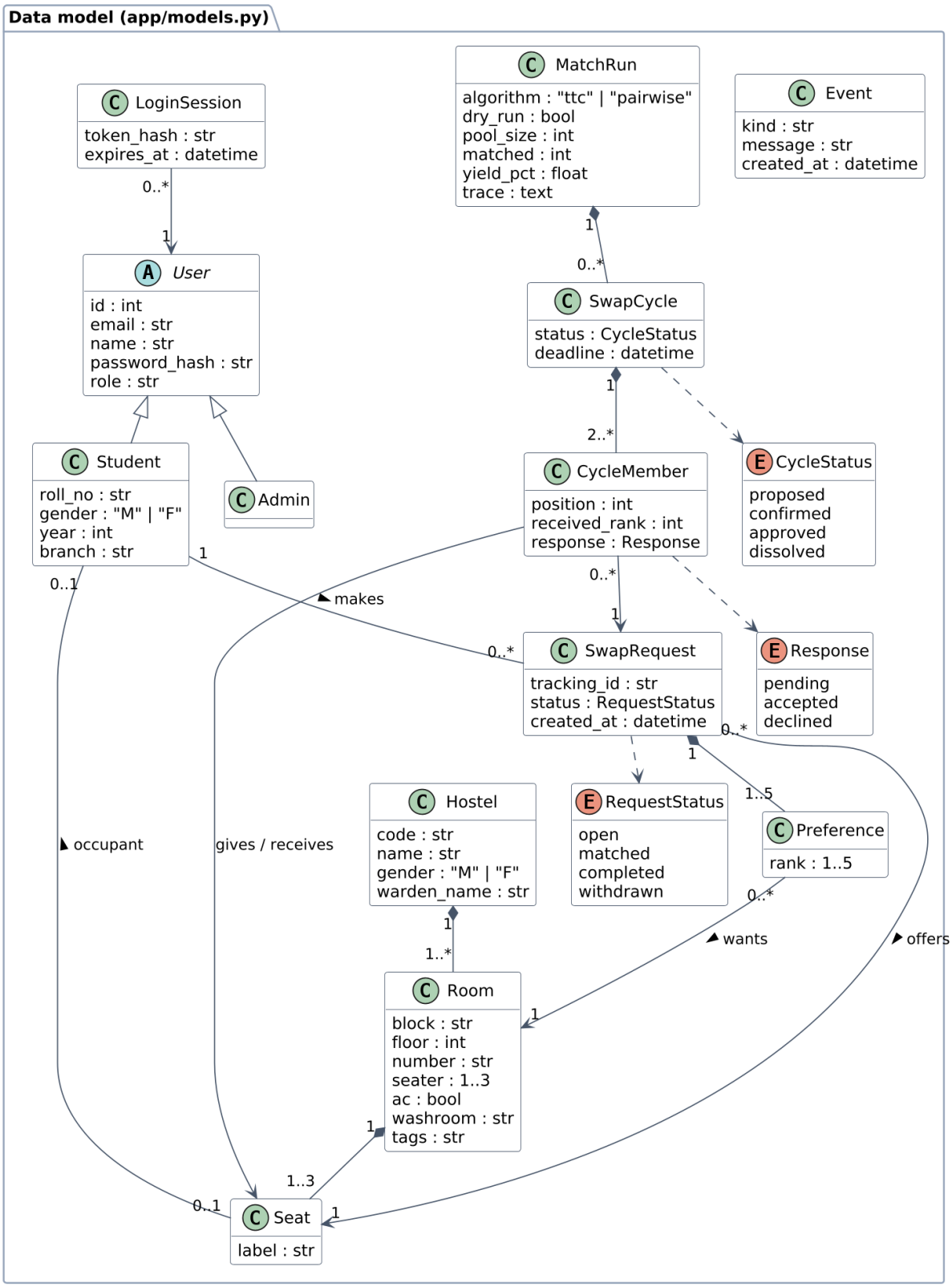


Figure B.1.: Class diagram, part 1: the domain model. Student and Admin inherit from User (hollow triangle); a filled diamond means the part cannot exist without its whole (a room without its hostel, a seat without its room).

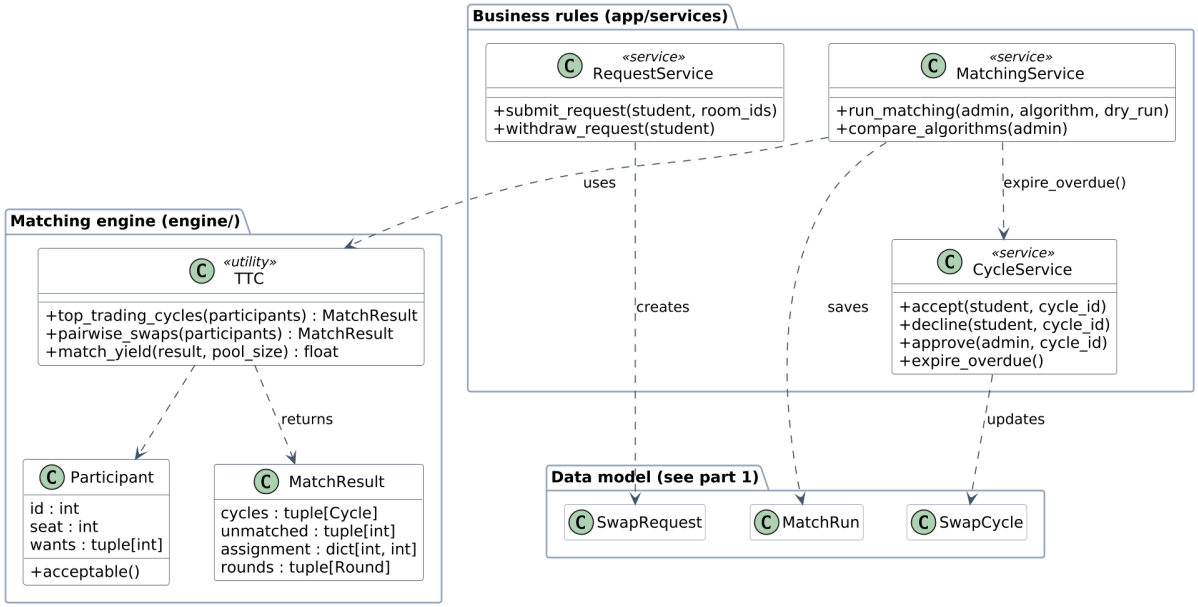


Figure B.2.: Class diagram, part 2: the services that hold the business rules, and the matching engine. Dashed arrows are dependencies: a class uses another without storing it. The three data classes are shown by name only; their attributes are in part 1.

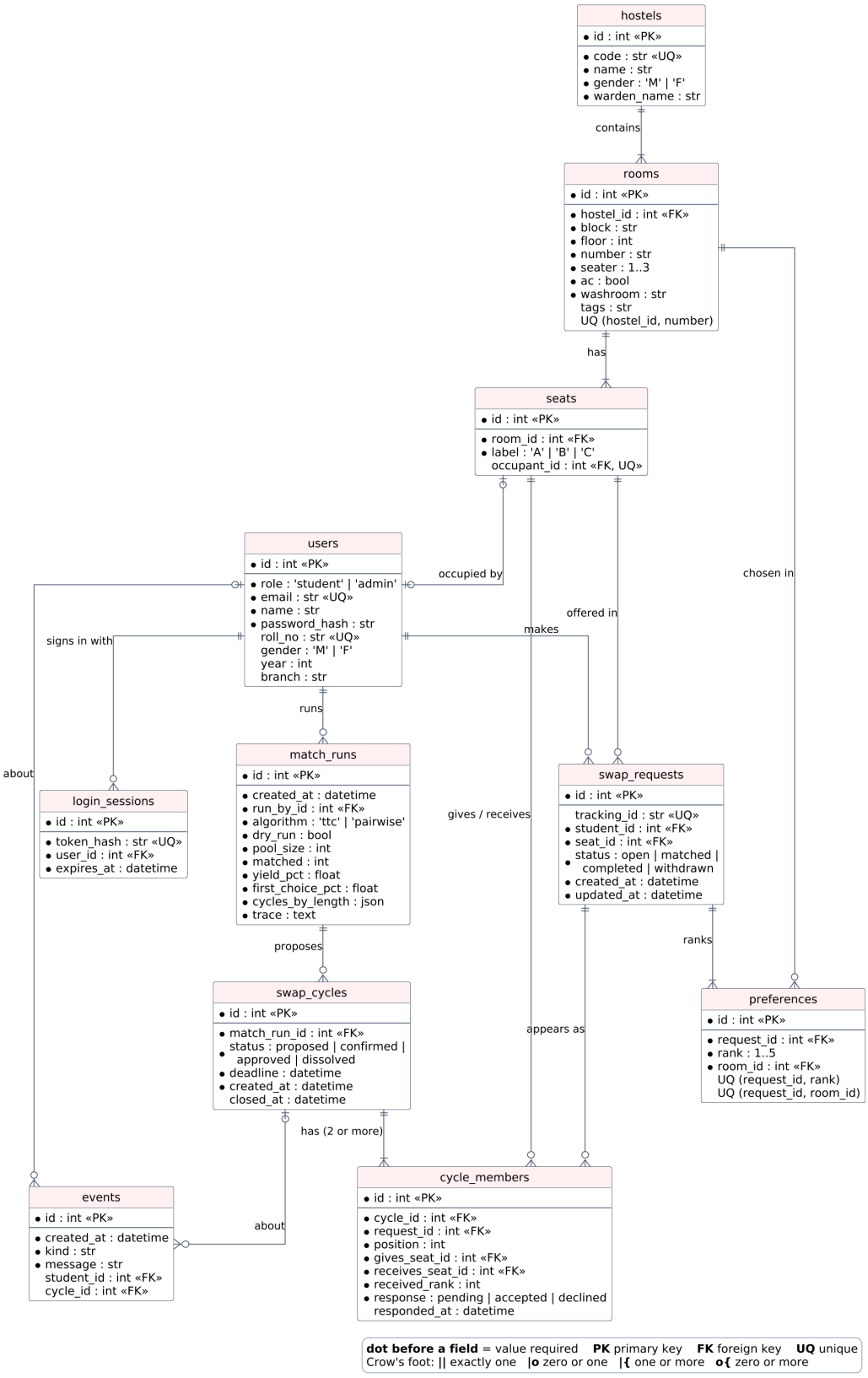


Figure B.3.: Entity–relationship diagram of the database, in crow's foot notation. It mirrors `code/backend/app/models.py`.

C. Traceability Matrix

Each built requirement is checked by at least one automated test in `code/backend/tests/`, or shown by a screenshot in this document. Paths starting with `app/` or `engine/`, and `seed.py`, are under `code/backend/`; paths starting with `js/` are under `code/frontend/public/`. Test names leave out the `test_` prefix. The full suite (89 tests) runs with `python -m pytest` from the repository root.

Table C.1.: Requirements to code and tests

Requirements	Code	Tests
FR-AUTH-01, 05	app/routers/auth.py, app/deps.py	test_api: login_sets_session_cookie_and_me_works
FR-AUTH-03	app/routers/auth.py	test_api: wrong_password_is_rejected, unknown_email_gets_the_same_answer
FR-AUTH-04	app/routers/auth.py	test_api: student_using_staff_tab_is_told_to_switch
FR-AUTH-06	app/routers/auth.py	test_api: logout_ends_the_session
FR-AUTH-07	app/deps.py	test_api: pages_need_login, student_cannot_use_admin_routes
FR-AUTH-08	js/login.js	Figure 3.1
FR-ROOM-01	app/services/views.py	test_views_service: rooms_listing_only_shows_own_gender_hostels
FR-ROOM-02 to 06	app/services/views.py, js/rooms.js	Figure 3.2
FR-REQ-01, 06	app/services/requests.py	test_requests_service: submit_saves_ranked_open_request_with_tracking_id
FR-REQ-02	js/rooms.js	Figure 3.2
FR-REQ-03	app/services/requests.py	test_requests_service: empty_list_rejected, more_than_five_rejected, duplicate_room_rejected
FR-REQ-04	app/services/requests.py	test_requests_service: student_without_seat_rejected, second_active_request_rejected
FR-REQ-05	app/services/requests.py	test_requests_service: unknown_room_rejected, own_room_rejected, room_in_other_gender_hostel_rejected

Requirements	Code	Tests
FR-REQ-07	app/services/requests.py	test_requests_service: withdraw_open_request, withdraw_without_request, withdraw_while_matched_is_refused; test_api: withdraw_over_http
FR-MATCH-01, 05	app/services/matching.py, engine/ttc.py	test_matching_service: seeded_pool_gives_one_2_3_and_4_way_chain, each_member_receives_the_next_members_seat
FR-MATCH-03	engine/ttc.py	test_ttc: double_room_seats_are_tried_in_listed_order, same_result_whatever_the_input_order
FR-MATCH-04	engine/ttc.py	test_ttc: random_pools_obey_the_basic_guarantees, nobody_wants_you_back_means_unmatched
FR-MATCH-06	app/services/matching.py	test_matching_service: dry_run_saves_the_run_but_changes_nothing
FR-MATCH-07	engine/metrics.py, app/services/matching.py	test_metrics: all three tests; test_matching_service: trace_explains_every_chain_in_plain_words
FR-MATCH-09	engine/pairwise.py, app/services/matching.py	test_pairwise: all five tests; test_matching_service: pairwise_comparison_matches_fewer_than_ttc, pairwise_cannot_save_real_swaps
FR-CONF-01	js/dashboard.js, js/ring.js	test_views_service: dashboard_shows_the_swap_chain_after_matching; Figure 3.4
FR-CONF-02	app/services/cycles.py	test_cycles_service: everyone_accepting_confirms_the_cycle
FR-CONF-03	app/services/cycles.py	test_cycles_service: one_decline_dissolves_and_returns_others_to_pool
FR-CONF-04	app/services/cycles.py	test_cycles_service: deadline_dissolves_unanswered_cycles, answering_after_deadline_is_refused
FR-CONF-05	app/services/cycles.py	test_cycles_service: outsider_cannot_answer, answering_twice_is_refused
FR-APPR-01, 02	app/services/cycles.py	test_cycles_service: approve_needs_everyone_confirmed, approve_moves_students_into_their_new_seats
FR-TRACK-01, 03	app/services/views.py	test_views_service: dashboard_without_request, dashboard_with_open_request, dashboard_after_approval_shows_new_room

Requirements	Code	Tests
FR-TRACK-04, 05	app/services/views.py	test_views_service: admin_overview_reflects_latest_run, admin_requests_rows
FR-TRACK-06	app/services/views.py	test_views_service: export_csv_has_header_and_one_row_per_request
FR-TRACK-07	app/services/views.py	test_views_service: public_stats_show_real_counts; test_api: public_stats_need_no_login
NFR-PERF-01	engine/ttc.py	test_ttc: two_thousand_students_run_quickly
NFR-PERF-03	all four pages	Figure 3.5
NFR-SEC-01, 02	app/security.py	test_security: all five tests
NFR-SAFE-05	seed.py	test_seed: all four tests

D. To Be Determined List

TBD-1 The source of real hostel, room and occupancy data for a pilot, and who keeps it up to date.
TBD-2 Approval to use the institute's Microsoft sign-in (`FR-AUTH-11`).
TBD-3 Where the system will be hosted, and who runs it.
TBD-4 Whether 48 hours is the right confirmation deadline for the hostel office.
TBD-5 How often the hostel office wants rounds to run (`FR-MATCH-10`).
TBD-6 The format the hostel office needs for the signed approval form (`FR-APPR-03`).